INSTITUTO UNIVERSITÁRIO DE LISBOA

Relatório

Armazenamento Para Big Data

2º ano da Licenciatura de Ciência de Dados

1º semestre | 2025/2026

10 de dezembro de 2025

Docente

Pedro Ramos

Realizado por

Gonçalo Ferreira | Nº130630

Rafael Azevedo | Nº130617

Rodrigo Silva | Nº130637

Índice

1. Introdução……..3

- *a) Contexto do Projeto (ATP Tennis Professionals Dataset).....3*
- *b) Objetivos do Trabalho.....3*

2. Base de Dados e Importação para MongoDB……...4

- *a) Conteúdo das Colunas e Atributos.....4*
- *b) Importação dos Dados (mongoimport).....4*

3. Criação de Coleções…….……..5

- *a) Coleção player.....5*
- *b) Coleção tournaments.....5*
- *c) Coleção games.....5*

4. Tratamento de Dados para as Coleções no MongoDB……6

- *a) Limpeza e Pré-processamento Inicial.....6*
  - *i) Coleção player.....6*
  - *ii) Coleção tournaments.....8*
  - *iii) Coleção games.....9*
- *b) Exportação para csv (mongoexport).....11*

5. Criação das tabelas em MySQL e Modelação da Base de Dados Relacional……...12

- *a) Importação dos ficheiros csv para o MySQL.....12*
- *b) Limpeza e Tratamento Final dos Dados.....12*
- *c) Criação das Chaves Primárias e Estrangeiras.....16*
- *d) Modelo Relacional.....18*

6. Análise de Resultados e Queries……..19

- *a) Estatísticas por País (Jogadores, Torneios, Rondas).....19*
- *b) Top 10 Jogadores por % de Vitórias.....20*
- *c) Top 10 Jogadores Canhotos em Grand Slams.....20*
- *d) Top 5 Jogadores em Piso Duro (Hard Ground).....21*

1) Introdução

O presente relatório descreve o trabalho prático desenvolvido no âmbito da unidade curricular de Ciência de Dados, focando-se na manipulação, transformação e análise de dados desportivos profissionais. O projeto percorre todo o ciclo de vida dos dados, desde a sua importação num sistema NoSQL (MongoDB), passando pelo seu tratamento e limpeza, até à sua modelação e interrogação numa base de dados relacional (MySQL).

a) Contexto do Projeto (ATP Tennis Professionals Dataset)

O objeto de estudo deste trabalho é o "ATP Tennis Professionals Dataset". Os dados utilizados foram recolhidos a partir do site oficial da ATP Tour (www.atptour.com) e dizem respeito a todas as partidas de tenistas masculinos.

O dataset foca-se especificamente nos jogadores do "Top 500" que competiram entre 28 de março de 1973 e 14 de fevereiro de 2022. A informação foi fornecida originalmente em formato JSON, através do ficheiro atpplayers.json, contendo registos detalhados que incluem a identificação do jogador (nome, altura, mão dominante e mão de apoio, link do jogador), detalhes do torneio (nome, localização, data, tipo de piso, prémio) e o resultado jogo a jogo.

b) Objetivos do Trabalho

Importação e Limpeza: Importar os dados brutos para o MongoDB para realizar uma limpeza e pré-processamento inicial, exportando-os posteriormente para formato CSV.

Modelação de Dados: Transformar a estrutura plana original num modelo relacional normalizado, definindo tabelas para jogadores, torneios e jogos, bem como as respetivas chaves primárias e estrangeiras.

Análise de Dados (SQL): Demonstrar a eficácia da base de dados criada através da execução de quatro queries SQL complexas para responder a questões de negócio, tais como:

- Estatísticas por país (número de jogadores, torneios e rondas).
- Identificação do Top 10 de jogadores por percentagem de vitórias.
- Análise de jogadores esquerdinos em torneios do Grand Slam.

- Desempenho dos melhores jogadores em piso duro (Hard Ground).

2) Base de Dados e Importação para MongoDB

a) Conteúdo das Colunas e Atributos

Atributos	***Conteúdo***
_id	Identificador de cada linha
PlayerName	Nome do jogador profissional de ténis (ex: "Novak Djokovic")
Born	País e cidade de nascimento do jogador
Height	Altura do jogador em centímetros
Hand	Mão dominante do jogador (ex: "Right-Handed, Two-Handed Backhand")
LinkPlayer	Link identificador de cada jogador do site da ATP
Tournment	Nome do torneio onde o jogo ocorreu
Location	Cidade e país onde se realizou o torneio
Date	Data de referência do jogo ou torneio (formato YYYY.MM.DD)
Ground	Tipo de superfície do campo (ex: "Hard", "Clay", "Grass")
Prize	Valor do prémio monetário associado
GameRound	Ronda do torneio a que corresponde o jogo
GameRank	Ranking do jogador no momento do jogo
Oponent	Nome do adversário
WL	Indicador de Vitória (*W - Win*) ou Derrota (*L - Loss*)
Score	Resultado final dos sets da partida (ex: "61 36 16")

b) Importação dos Dados (mongoimport)

Iniciámos a importação do ficheiro disponibilizado no Moodle, atpplayers.json, para o MongoDB, utilizando o seguinte código: *"**mongoimport --username root --password root --authenticationDatabase admin --db atpplayers --collection players --drop --file /tmp/import/atpplayers.json**"*.

O processo consistiu na criação de uma base de dados designada atpplayers e de uma coleção inicial chamada players, conforme os requisitos do projeto. O comando executado no terminal garantiu que todos os documentos contidos no ficheiro atpplayers.json fossem

inseridos corretamente na coleção alvo, ficando disponíveis para as fases de limpeza e tratamento de dados.

3) Criação de Coleções

Procedeu-se à criação de três coleções, essenciais para a facilitação das questões propostas pelos docentes: as coleções 'player', 'tournaments' e 'games', implementadas em MongoDB. Para esse efeito, foram utilizados os seguintes códigos:

a) *Coleção player*

db.players.aggregate([{$group:{_id:"$LinkPlayer",Name:{$first:"$PlayerName"},Born:{$first:"$Born"},Height:{$first:"$Height"},Hand:{$first:"$Hand"},PlayerLink:{$first:"$LinkPlayer"}}},{$project:{_id:0,PlayerLink:{$replaceAll:{input:"$PlayerLink",find:"/",replacement:""}},Name:"$Name",Born:"$Born",Height:"$Height",Hand:"$Hand"}},{$merge:"player"}]);

Dado que o atributo *LinkPlayer* funciona como o identificador único de cada jogador no torneio, optou-se por agregar as diversas linhas através desse atributo, empregando a função **$group** e definindo o *LinkPlayer* como _**id**. Considerando que a tabela 'player' deve conter um registo para cada jogador, procedeu-se à seleção exclusiva da primeira linha de cada grupo, utilizando a funcionalidade **$first**.

b) *Coleção tournaments*

db.players.aggregate([{$project:{Filter:{$concat:["$Tournament","","$Date"]},Name:"$Tournament",Location:"$Location",Date:"$Date",Ground:"$Ground",Prize:"$Prize"}},{$group:{_id:"$Filter",Name:{$first:"$Name"},Location:{$first:"$Location"},Date:{$first:"$Date"},Ground:{$first:"$Ground"},Prize:{$first:"$Prize"}}},{$project:{'_id':0}},{$merge:"tournaments"}]);

A distinção dos torneios foi efetuada através da utilização do nome e da data de cada um para a aplicação da filtragem. Considerando a necessidade de a coleção '*tournaments*' possuir uma entrada para cada torneio, recorreu-se à funcionalidade **$first** para selecionar apenas a primeira linha de cada agrupamento.

c) *Coleção games*

db.players.aggregate([{$project:{_id:0,Player:"$PlayerName",Oponent:"$Oponent",Tournament:"$Tournament",Date:"$Date",GameRound:"$GameRound",Ground:"$Ground",GameRank:"$GameRank",WL:"$WL",Score:"$Score"}},{$merge:"games"}]);

Ao contrário das outras, nesta coleção não foi necessária a função **$first** pois é essencial ter todos os jogos e não apenas o primeiro.

4) Tratamento de Dados para as Coleções no MongoDB

a) *Limpeza e Pré-processamento Inicial*

Depois de criadas e analisadas todas as coleções, o grupo detetou algumas inconsistências. Por essa razão, tornou-se essencial proceder a um tratamento e limpeza minuciosos de cada uma delas, com o objetivo de corrigir quaisquer erros.

i) Coleção player

Para executar o comando SELECT e obter o número de jogadores por país, foi necessário realizar um tratamento e limpeza da coluna 'Born'. Relativamente à coluna 'Hand', também foi essencial tratar e limpar essa informação, sendo posteriormente dividida em duas colunas no SQL, referido mais tarde no relatório.

Foram identificadas **2221 localidades distintas** na coluna 'Born'. Dessas, **541 contêm apenas a cidade ou o país** de origem do jogador.

Adicionalmente, **6276 jogadores** (correspondendo a 63% do total de jogadores na coleção) **não possuem qualquer informação** preenchida no campo 'Born' relativa à sua cidade/país de origem.

```
atpplayers> db.player1.count({"Born":''}, {})
6276
```

Antes da resolução do problema, o grupo optou por **criar a coluna "id_player"**. Essa coluna foi gerada a partir de um código específico presente na coluna "PlayerLink", **com o objetivo de evitar a atribuição incorreta de países a jogadores com o mesmo nome**.

Para resolver este problema, o grupo optou por desenvolver um código em Python (web scraping). Este código tinha como função pesquisar a nacionalidade de todos os jogadores que tinham o campo 'Born' vazio. O resultado dessa pesquisa seria um ficheiro de texto (updates_born.txt) contendo todos os updates necessários para preencher e corrigir essa coluna. No entanto, mesmo com este método, **317 jogadores** (≈3%) **continuaram sem ter informação sobre o seu país** (104 com valor null, 213 com valor vazio).

Adicionalmente, a coluna 'Born' apresenta os mesmos países escritos de forma diferente, tal como o país "Brazil" aparece como "BRA" ou "Brasil".

Para a criação da coleção player_borncountry, foram inicialmente selecionadas todas as combinações distintas da coluna 'Born'. Em seguida, utilizou-se a função ***split*** para extrair o país, utilizando a vírgula como delimitador. Os registos que continham apenas o nome do país ou cidade foram posteriormente processados/ajustados para garantir a consistência das colunas. O código utilizado foi o seguinte:

db.player.aggregate([{$project: {pl_city_country: "$Born",Country: {$arrayElemAt: [{ $split: ["$Born", ","] }, -1]}}},{$group: {_id: {pl_city_country: "$pl_city_country",Country: "$Country"}}},{$project: {_id: 0,pl_city_country: "$_id.pl_city_country",Country: "$_id.Country"}},{$merge: "player_borncountry"}]);

Foram assim importadas 2369 linhas.

```
atpplayers> db.player_borncountry.find().count()
2369
```

Observou-se também que, em muitos desses países, existia um espaço em branco antes do nome, o que poderia dificultar a futura limpeza dos dados. Procedeu-se, então, à remoção desses espaços em branco no início (e também no final) e manteve-se apenas a última parte da *string* após a última vírgula, resultando na modificação de **1676 valores**.

```
atpplayers> db.player_borncountry.updateMany({},
... [{$set:{Country:{$trim:{input:{$arrayElemAt:[{$split:["$Country",', ']},-1]}}}}}]);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2369,
  modifiedCount: 1676,
  upsertedCount: 0
}
```

De seguida, **procedemos à limpeza da tabela player_borncountry**. Foram efetuadas todas as atualizações necessárias para corrigir os países mal escritos ou aqueles em que apenas constava a cidade. O grupo utilizou o MongoDB para realizar esta limpeza, o que se revelou um processo mais rápido e eficaz. As atualizações necessárias foram iniciadas com o recurso à função **updateMany**.

Após o tratamento e a limpeza da coluna 'Born', o grupo apercebeu-se de que a coluna 'Hand' tinha **3987 registos com valores 'null'**.

```
atpplayers> db.player1.count({"Hand":'null'}, {})
3987
```

Para tal, o grupo executou o mesmo código em Python (web scraping), originalmente concebido para procurar os valores da coluna 'Born', mas desta vez para a coluna 'Hand'.

Após a execução do ficheiro txt (updates_hand.txt) com todas as atualizações, **os valores 'null' foram modificados**, **à exceção de 203 registos** (≈2%).

Os demais códigos, desenvolvidos em MySQL, para a limpeza e tratamento desta coluna, serão detalhados mais à frente.

ii) Coleção Tournaments

Verificou-se a necessidade de proceder à limpeza da coluna 'Location' na tabela tournaments, a qual contém a informação relativa ao local de realização do torneio de ténis.

O processo para a limpeza e tratamento desta coluna é semelhante com a da coluna 'Born' da tabela player, com a diferença de que agora, não existem valores nulos nem valores vazios.

Para determinar a extensão da limpeza necessária, o grupo executou comandos para verificar o número total de locais distintos, identificar quantos desses locais tinham apenas a cidade ou apenas o país do torneio.

Notou-se haver **2512 localidades diferentes na coluna**, das quais **219 destes valores** apresentam na coluna **apenas a cidade ou país do torneio**.

```
atpplayers> db.tournaments.aggregate([
...     {
...         $group: {
...             _id: null,
...             uniqueLocations: { $addToSet: "$Location" }
...         }
...     },
...     {
...         $project: {
...             _id: 0,
...             distinctCount: { $size: "$uniqueLocations" }
...         }
...     }
... ])
[ { distinctCount: 2512 } ]
```

Com o intuito de auxiliar na padronização da coluna 'Location', foi concebida uma coleção auxiliar denominada location_tournaments. Esta coleção é composta por duas colunas: 'City_country', que contém os valores singulares da coluna 'Location' original e serve como elemento de ligação com a coleção tournaments, e 'Country', que armazena exclusivamente o país correspondente a cada localidade. Para o preenchimento dest a nova coleção, procedeu-se à inserção dos dados, selecionando as combinações distintas de

cidade-país presentes na coluna 'Location'. A identificação do país ('Country') foi realizada através do isolamento da última porção da *string* após a última vírgula.

db.players.aggregate([{$project: {City_country: "$Location", Country: {$arrayElemAt: [{$split: ["$Location", ","]}, -1]}}}, {$group: {_id: {City_country: "$City_country", Country: "$Country"}}}, {$project: {_id: 0, City_country: "$_id.City_country", Country: "$_id.Country"}}, {$merge: "location_tournaments"}]);

Reparou-se, à semelhança da coluna 'Born' inicialmente não tratada, que em alguns destes países aparece um espaço em branco antes do país, o que iria dificultar a limpeza dos dados futuramente. Foi então retirado este espaço em branco através do seguinte comando, afetando **2293 linhas**:

```
db.location_tournaments1.updateMany({}, [{$set:{Country:{$trim:{input:{$arrayElemAt:[{$split:["$Country",', ']},-1]}}}}}])
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 5024,
  modifiedCount: 2293,
  upsertedCount: 0
}
```

Em seguida, procedemos à limpeza da tabela location_tournaments, realizando todas as atualizações necessárias para corrigir t:,os países que estavam mal escritos ou onde apenas a cidade estava indicada. Para essa limpeza, o grupo utilizou o MongoDB, o que se revelou um processo mais rápido e eficaz. As atualizações foram iniciadas através da função updateMany.

Os demais códigos, desenvolvidos em MySQL para esta coluna com o intuito de a tornar mais limpa e tratada, serão detalhados mais à frente.

iii) Coleção games

Para permitir alterações seguras, foi estabelecida uma coleção secundária, denominada games2. Esta coleção não só serve como ambiente de trabalho para as modificações, mantendo a coleção original, games, como cópia de segurança, mas também serve para uma limpeza de alguns dados duplicados.

Para viabilizar a análise estatística referente ao tipo de piso usado nas partidas, a coluna 'Ground' foi submetida a um processo de limpeza, uma vez que apresentava valores omissos.

A solução adotada consistiu em pesquisar na tabela games os torneios onde a informação sobre o tipo de piso estava ausente (**identificaram-se 191 partidas sem essa informação**).

```
atpplayers> db.games.find({Ground: ""}).count();
191
```

Recorremos subsequentemente a uma pesquisa na internet para obter dados que permitissem alterar alguns dos valores nulos, utilizando a função **updateMany** do MongoDB. Desta forma, o comando foi aplicado apenas aos torneios "M15 Antalya" que tem 61 jogos sem Ground, e "M15 Cancun", "M15 Opava" que têm ambos 60 jogos sem Ground, sobrando assim 10 jogos sem Ground.

Relativamente à coluna 'WL', o grupo notou durante a sua análise que, para além dos valores intuitivos W = Win ou L = Loss, esta apresenta também valores vazios. A coluna 'Oponent', para além do nome dos oponentes, apresenta também o valor 'bye' (jogo que não aconteceu por falta de comparência ou desistência do adversário e o jogador passa para a próxima ronda). Existiam 19173 registos em que a coluna 'WL' tinha valores vazios e 19155 registos em que a coluna 'Oponent' tinha valores "bye", sendo que 19152 registos têm os dois valores em comum.

Criou-se uma coleção games_filtered para armazenar os registos que não vão ser usados para fins estatísticos, nos quais estão incluídos os registos com a coluna 'Oponent':'bye', a coluna 'WL' com valores vazios e os quatro jogos duplicados mas com o score diferente (explicado mais à frente no relatório).

Após a criação desta coleção já podemos apagar estes registos da coleção games2 sem comprometer a perda dos dados.

Depois da eliminação dos registos, criam-se duas colunas (vencedor e perdedor) para auxiliar na remoção de jogos duplicados.

As colunas 'winner' e 'loser' foram criadas para normalizar os dados dos jogos.

Como os registos originais podem conter o mesmo jogo duas vezes (uma como vitória e outra como derrota, invertendo Player/Oponent), a criação destas colunas permite identificar de forma inequívoca o vencedor e o derrotado. Isto possibilita a deteção e remoção de duplicados espelhados e garante a integridade e consistência do dataset.

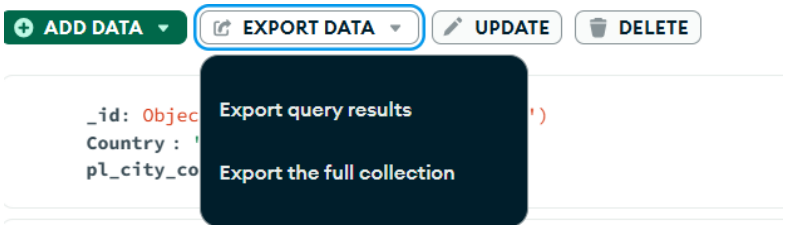
Depois da criação destas colunas partiu-se para a identificação e remoção dos jogos duplicados. Jogos duplicados caracterizam-se por terem a mesma data, nome do torneio, tipo de piso, ronda do torneio, vencedor e perdedor. Entre os dois duplicados, optámos por remover aqueles em que a coluna 'WL' tinha valor L, para ser mais fácil responder aos selects propostos. Utilizou-se o seguinte código:

const losersToDelete = db.games2.aggregate([{ $group: { _id: { Date: "$Date", Tounament: "$Tounament", Ground: "$Ground", GameRound: "$GameRound", winner: "$winner", loser: "$loser" }, docs: { $push: { _id: "$_id", WL: "$WL" } }, count: { $sum: 1 } } }, { $match: { count: { $gt: 1 } } }, { $project: { loserIds: { $map: { input: { $filter: { input: "$docs", as: "d", cond: { $eq: ["$$d.WL", "L"] } } }, as: "l", in: "$$l._id" } } } }, { $unwind: "$loserIds" }, { $project: { _id: "$loserIds" } }]).toArray(); if (losersToDelete.length > 0) { const ids = losersToDelete.map(d => d._id); const res = db.games2.deleteMany({ _id: { $in: ids } }); print("Removidos:", res.deletedCount); } else { print("Nenhum WL:L encontrado nos grupos duplicados."); }

Ao realizar este código, percebemos que havia ainda quatro grupos de duplicados, em que o que mudava era a coluna 'Score'. Fomos pesquisar as verdadeiras pontuações desses quatro jogos, e eliminámos aqueles que não eram verdadeiras. Anteriormente foi explicado que esses "falsos" duplicados, foram para a coleção games_filtered. O site utilizado para obter as verdadeiras pontuações foi o seguinte: https://www.daviscup.com/en.

b) Exportação para csv (mongoexport)

Após as alterações feitas no mongodb deu-se às coleções no Mongo Compass.



Procedeu-se à exportação das coleções *player*, *tournaments*, *games2*, *player_borncountry*, *location_tournamnets*, *games_filtered* e *players*. O método de exportação adotado consistiu em aceder a "EXPORT DATA", selecionar "Export query results" e, subsequentemente, optar por "Select fields in table". Nesta última secção, foi removida a coluna '_id' em todas as coleções. Por fim, selecionou-se a opção "CSV" para efetuar a exportação.

5) Criação das tabelas em MySQL e Modelação da Base de Dados Relacional

a) *Importação dos ficheiros csv para o MySQL*

Depois de alguns dos dados já estarem limpos e corretos e, de modo a começarmos a criar o modelo relacional, foi necessário construir e criar as tabelas player, tournaments e games na linha de comandos, após inserir os códigos iniciais (***mysql -u root -p','use atpplayers***).

Utilizámos o seguinte comando na linha de comandos para criar a tabela player com os atributos necessários. As variáveis selecionadas foram as mesmas que as empregues na criação da coleção player.

- ***create table player(Born varchar(150),Hand varchar(50),Height int(3),Name varchar(50),PlayerLink varchar(160),id_player varchar(4));***

  Posteriormente, inseriu-se a base de dados no PhpMyAdmin na tabela:

- **LOAD DATA INFILE '/var/lib/mysql-files/player.csv' INTO TABLE player FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 LINES (Born, Hand, @var_Height, Name, PlayerLink, id_player) SET Height = NULLIF(@var_Height, 'NA');**

Para as restantes coleções exportadas do MongoDB, foram utilizados códigos semelhantes, alterando apenas os nomes e os tipos das colunas.

Para além dos csv's importados do mongo, o grupo decidiu usar mais dois csv's externos para nos ajudar nos tratamentos e limpezas destas tabelas:

- (List of all countries with their 2 digit codes (ISO 3166-1) (retirado do enunciado)
- World Cities Database | Simplemaps.com

b) *Limpeza e Tratamento Final dos Dados*

Esta fase iniciou-se pela tabela countries, uma vez que, aquando da importação da coleção, foi necessário criar uma terceira coluna temporária designada 'erro' devido à

existência de linhas com três vírgulas. De seguida, foram executados códigos para atualizar os nomes e os códigos dos países. Por fim, a coluna 'erro' foi eliminada.

Exemplos dos códigos corridos:

- **update countries set Name="Bolivia, Plurinational State of" where Name="Bolivia";**
- **update countries set Code="BO" where Name="Bolivia, Plurinational State of";**
- **ALTER TABLE countries DROP COLUMN `erro`;**

Depois, partiu-se para a tabela player_borncountry em que se utilizou a tabela world_cities para se verificar que as cidades tinham sido atribuídas aos países corretos.

Exemplo dos códigos corridos:

- **update player_borncountry, world_cities set player_borncountry.Country = world_cities.country where player_borncountry.pl_city_country=world_cities.city;**
- **update player_borncountry, world_cities set player_borncountry.Country=world_cities.country where player_borncountry.pl_city_country=world_cities.country and not instr(pl_city_country,',')>0;**

Posteriormente, a coluna 'Country' foi criada na tabela player, onde serão inseridos os dados corretos e atualizados relativos aos países dos jogadores, com base na tabela player_borncountry. A coluna 'Born' permanece inalterada, contendo todas as informações sobre o local de nascimento do jogador.

- **alter table player add column Country Varchar(152) AFTER Born;**
- **update player, player_borncountry set player.Country = player_borncountry.Country where player.Born = player_borncountry.pl_city_country;**

Não obstante, o grupo verificou a existência de um conjunto de países nos quais a correspondência entre os países da tabela player_borncountry e countries não era idêntica. A título de exemplo, destacamos 'Taiwan', que na tabela countries corresponde a 'Taiwan, Province of China'. Neste seguimento, o grupo procedeu à identificação dessas exceções e, através do comando update, efetuou a respetiva correção.

Exemplos dos códigos corridos:

- **update player set Country = "Taiwan, Province of China" where Country = "Taiwan";**
- **update player set Country = "Venezuela, Bolivarian Republic of" where Country = "Venezuela";**

Dado que, futuramente, esta tabela terá de ser relacionada com a tabela countries através de chaves estrangeiras, optámos por substituir o valor do campo "Country" pelo valor do campo "code" da tabela countries, sempre que o campo "Country"(player) fosse coincidente com o campo "country"(countries).

- **update player, countries set player.Country = countries.code where player.Country = countries.Name;**

Relativamente à coluna 'Hand' da tabela player, a fim de obter estatísticas sobre os jogadores com as melhores mãos e tipos de pegas de raquete, foi necessário dividir esta coluna em duas, uma relativa à mão predominante do jogador e a outra relativa à 'backhand' do mesmo.

Deste modo, foi adicionada uma coluna à tabela players para indicar a *backhand*, mantendo a coluna existente ('Hand') para a mão predominante do jogador.

- **alter table player add column BackHand Varchar(152) after Hand;**
- **UPDATE player SET BackHand = TRIM(SUBSTRING_INDEX(Hand, ',', -1));**
- **UPDATE player SET Hand = SUBSTRING_INDEX(Hand, ',', 1);**

Finalmente, o grupo identificou que a coluna 'Height' continha algumas inconsistências, nomeadamente alturas consideradas anormais, como 3 cm. No entanto, visto que esta coluna não seria utilizada para análises estatísticas futuras, optou-se por não proceder à sua limpeza e tratamento.

De seguida, a tabela location_tournaments foi comparada com a tabela world_cities: se a primeira coluna de ambas as tabelas fosse coincidente, o respetivo país era adicionado à segunda coluna da tabela location_tournaments.

- **update location_tournaments , world_cities set location_tournaments.Country=world_cities.country where location_tournaments.City_country=world_cities.city;**

No comando seguinte, as mesmas tabelas foram comparadas. A coluna 'City_country' da tabela location_tournaments com a segunda coluna 'country' da tabela world_cities: se apenas existir o país na primeira coluna do 'country', este seria inserido na segunda coluna 'Country' na tabela location_tournaments. No comando subsequente, as mesmas tabelas foram sujeitas a uma comparação. A coluna 'City_country' da tabela location_tournaments foi confrontada com a segunda coluna 'country' da tabela world_cities: caso o país existisse apenas na primeira coluna ('country'), este seria inserido na segunda coluna 'Country' da tabela location_tournaments..

- **update location_tournaments, world_cities set location_tournaments.Country=world_cities.country where location_tournaments.City_country=world_cities.country and not instr(City_country,',')>0;**

De seguida, verificou-se a extrema importância de incluir a coluna 'País' na tabela torneios. Esta coluna foi devidamente preenchida e atualizada com os países correspondentes aos jogadores, obtendo-se esta informação a partir da tabela localizacao_torneios. A coluna 'Localização' permanece inalterada, preservando todas as informações relativas ao local de nascimento dos jogadores.

- **alter table tournaments add column Country Varchar(152) after Location;**
- **update tournaments,location_tournaments set tournaments.Country=location_tournaments.Country where tournaments.Location=location_tournaments.City_country;**

Mesmo após os comandos acima executados, verificaram-se algumas exceções que não foram abrangidas por estas atualizações devido a discrepâncias de caracteres entre programas ou, inclusive, pela insuficiência de dados nas tabelas referentes a determinados países/cidades, o que exigiu a realização de novos updates. Somente através deste procedimento foi possível manter a coerência e a integridade dos dados para efeitos de comparação.

Exemplos dos códigos corridos:

- **update tournaments set Country="Côte d'Ivoire" where Country like "%Côte %";**
- **update tournaments set country = "Lao People's Democratic Republic" where country = "Laos";**

Após uma reanálise dos valores que se encontravam inconsistentes com a tabela countries, foi identificado um registo de torneio assinalado como 'TBA'. Procedeu-se a uma investigação para determinar o significado da sigla, concluindo-se que 'TBA' corresponde a "To Be Announced". Posteriormente, e após nova pesquisa, o país onde o torneio efetivamente ocorreu foi devidamente registado.

Os comandos efetuados para fazer o update respetivo em MySQL foi:

- **update tournaments set Country="Botswana" where Name="Botswana F1";**
- **update tournaments set Country="Egypt" where Name="Egypt F1" or Name="Egypt F2" or Name="Egypt F3";**

Uma vez que, futuramente, teremos de relacionar esta tabela com a tabela countries através de chaves estrangeiras, optámos por substituir o valor do campo "Country" pelo valor do campo "code" da tabela countries, sempre que o campo "Country" da tabela tournaments fosse igual ao campo "country".

- **update tournaments , countries set tournaments.Country = countries.code where tournaments.Country = countries.Name;**

O tratamento e limpeza da tabela tournaments foram assim concluídos. O grupo optou por não realizar a limpeza e tratamento dos dados das colunas ‘date, prize, score e GameRank’, visto que não seriam utilizados para análises e estatísticas futuras.

Uma vez que se tornava necessário associar informações dos jogos e dos jogadores entre as tabelas player e games, recorrendo a chaves primárias e estrangeiras, o grupo decidiu criar uma coluna, 'id_player', que estabelece a associação entre o nome do jogador ('Player') e o seu ID (conforme realizado na tabela player). Para o efeito, foram criados os seguintes índices com o intuito de tornar o comando mais rápido e eficiente, dada a manipulação de um volume significativo de dados:

- **alter table games add column id_player VARCHAR(10); alter table games add index(Player, id_player); alter table player add index (Name, id_player); update games, player set games.id_player = player.id_player where games.Player = player.Name; alter table games drop index Player; alter table player drop index Name;**

c) Criação das Chaves Primárias e Estrangeiras

Foi então posto, como referido anteriormente, o atributo 'id_player' como chave primária da tabela player:

- **ALTER TABLE player ADD PRIMARY KEY(id_player);**

Visto que esta base de dados será utilizada para fins estatísticos e analíticos, é imperativo assegurar a consistência, coerência e integridade dos dados. Isto implica a deteção de violações de regras de integridade, nomeadamente ao nível das chaves primárias e estrangeiras, tornando a criação de chaves estrangeiras de importância fundamental. Adicionalmente, estas chaves são sempre valiosas para a compreensão do dicionário de dados. Neste contexto, o atributo 'Country' foi definido como chave estrangeira, estabelecendo a ligação com a tabela countries:

- **alter table player add foreign key (Country) REFERENCES countries(code) on**
- **delete restrict on update cascade;**

Para a tabela countries, foi colocada como chave primária o código de cada país ('code'), uma vez que seria a única possível para posteriormente se relacionar com as demais tabelas.

- **alter table countries add primary key(code);**

Uma vez que a tabela tournaments foi construída tendo em maior conta o nome e a data de realização do torneio, a chave primária foi feita com base nestes dois atributos:

- **alter table tournaments add primary key (Name,Date);**

O atributo 'Country' foi definido como uma chave estrangeira, estabelecendo a ligação à tabela countries pelo código:

- **alter table tournaments add foreign key (Country) references countries(code) on delete restrict on update cascade;**

Na implementação da chave primária da tabela games, o grupo fez uso do comando de **auto_increment**. Foi necessário criar um id para cada uma das linhas pois cada uma delas referia-se a um jogo diferente:

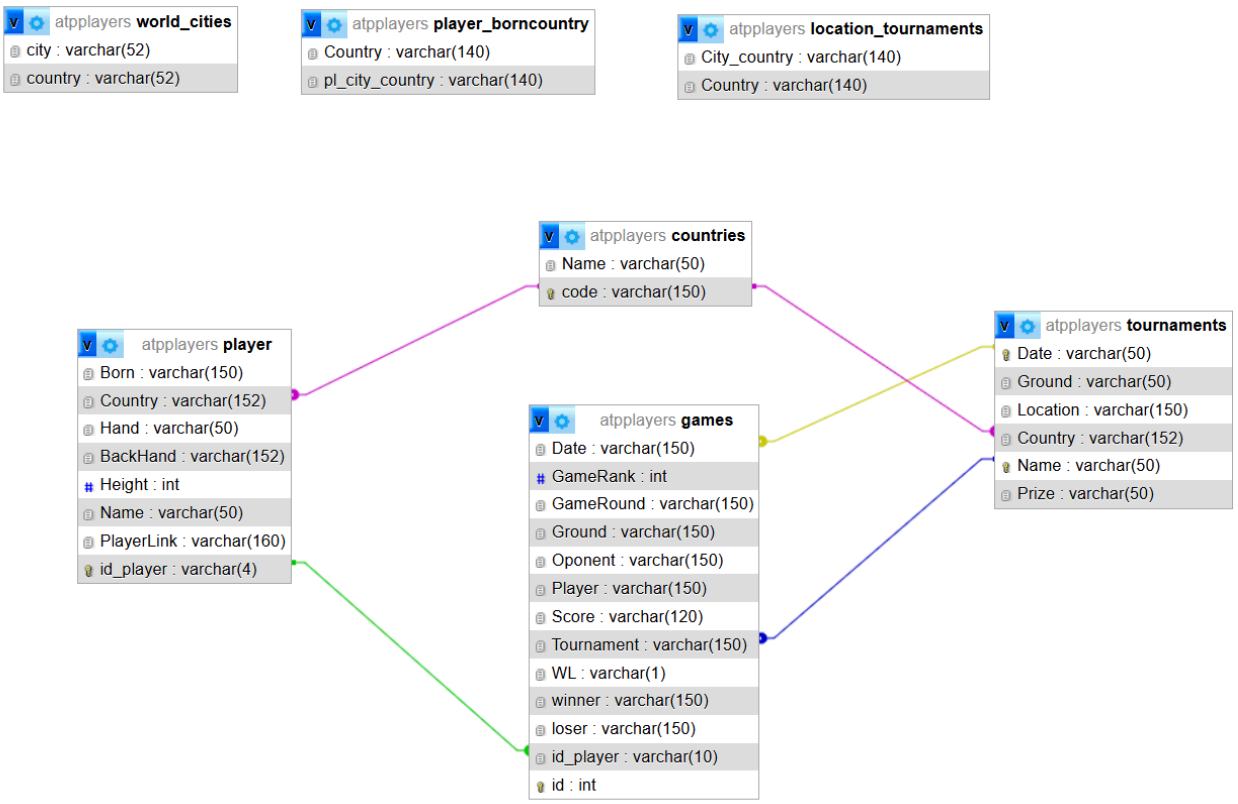
- **alter table games add id int not null auto_increment primary key;**

Uma vez que esta tabela tem em comum o id_player pela tabela player, o nome do torneio jogado e a sua data pela tabela tournament, estes atributos foram impostos como chaves estrangeiras. Para ajudar, foram criados índices para acelerar o processo:

- **ALTER TABLE games ADD FOREIGN KEY (id_player) REFERENCES player(id_player) on delete restrict on update cascade;**
- **alter table tournaments add index (Date);**
- **ALTER TABLE games ADD FOREIGN KEY (Tournament, Date) REFERENCES tournaments (Name, Date)ON DELETE RESTRICT ON UPDATE CASCADE;**
- **ALTER TABLE `tournaments` DROP INDEX `Date`;**

d) Modelo Relacional

O modelo relacional, após a imposição de todas as chaves primárias e estrangeiras, ficou disposto da seguinte maneira:



6) Análise de Resultados e Queries

Antes de se começar a fazer os comandos select decidiu-se criar índices para que se fossem mais rápidos e eficazes.

CREATE INDEX idx_player_name ON player(Name);

CREATE INDEX idx_game_winner ON games(winner);

CREATE INDEX idx_game_loser ON games(loser);

CREATE INDEX idx_tournament_name ON tournaments(Name);

CREATE INDEX idx_game_tournament ON games(Tournament);

a) *Estatísticas por País (Jogadores, Torneios, Rondas)*

```
SELECT
    c.Name,
    COUNT(DISTINCT p.id_player) AS TotalPlayers,
    COUNT(DISTINCT t.name) AS TotalTournaments,
    COUNT(DISTINCT g.id) AS TotalGames
FROM countries c, player p, tournaments t, games g
WHERE p.id_player = g.id_player
  AND g.Tournament = t.Name
  AND t.Country = c.code
  AND t.country IS NOT NULL
GROUP BY c.Name
LIMIT 10;
```

Name	TotalPlayers	TotalTournaments	TotalGames
Algeria	243	9	914
Andorra	145	3	349
Argentina	1294	110	14256
Armenia	74	2	216
Aruba	60	2	124
Australia	1686	123	23147
Austria	1804	78	11491
Azerbaijan	86	6	277
Bahamas	48	20	118
Bahrain	156	5	374

b) *Top 10 Jogadores por % de Vitórias*

```
mysql> SELECT
    ->     p.Name,
    ->     -- Conta quantas vezes o nome aparece na coluna winner
    ->     SUM(CASE WHEN g.winner = p.Name THEN 1 ELSE 0 END) AS WinningGames,
    ->     -- O total  o nmero de linhas que este JOIN encontrou (pois ele apanha vitrias e derrotas)
    ->     COUNT(*) AS TotalGames,
    ->     -- Percentagem
    ->     (SUM(CASE WHEN g.winner = p.Name THEN 1.0 ELSE 0.0 END) / COUNT(*)) * 100 AS WinPercentage
    -> FROM
    ->     player p
    -> JOIN
    ->     games g ON (p.Name = g.winner OR p.Name = g.loser)
    -> GROUP BY
    ->     p.id_player, p.Name
    -> ORDER BY
    ->     WinPercentage DESC
    -> LIMIT 10;
+---------------+-------------+------------+---------------+
| Name          | WinningGames | TotalGames | WinPercentage |
+---------------+-------------+------------+---------------+
| Taras Beiko   |           1 |          1 |     100.00000 |
| Rafael Nadal  |        1130 |       1360 |      83.08800 |
| Novak Djokovic |        1059 |       1277 |      82.92900 |
| Bjorn Borg    |         659 |        805 |      81.86300 |
| Roger Federer |        1281 |       1568 |      81.69600 |
| Ivan Lendl    |        1079 |       1322 |      81.61900 |
| John McEnroe  |         891 |       1093 |      81.51900 |
| Jimmy Connors |        1281 |       1584 |      80.87100 |
| Tomasz Iwanski |           4 |          5 |      80.00000 |
| James Tracy   |           4 |          5 |      80.00000 |
+---------------+-------------+------------+---------------+
10 rows in set (5.507 sec)
```

c) *Top 10 Jogadores Canhotos em Grand Slams*

```
mysql> SELECT
    ->     p.Name,
    ->     SUM(CASE WHEN g.winner = p.Name THEN 1 ELSE 0 END) AS WinningGames,
    ->     COUNT(*) AS TotalGames,
    ->     (SUM(CASE WHEN g.winner = p.Name THEN 1.0 ELSE 0.0 END) / COUNT(*)) * 100 AS WinPercentage
    -> FROM
    ->     player p
    -> JOIN
    ->     games g ON (p.Name = g.winner OR p.Name = g.loser)
    -> JOIN
    ->     tournaments t ON g.Tournament = t.Name
    -> WHERE
    ->     p.Hand LIKE '%Left%'
    ->     AND t.Name IN ('Wimbledon', 'US Open', 'Australian Open', 'Roland Garros')
    -> GROUP BY
    ->     p.id_player, p.Name
    -> ORDER BY
    ->     WinPercentage DESC
    -> LIMIT 10;
+-----------------+-------------+------------+---------------+
| Name            | WinningGames | TotalGames | WinPercentage |
+-----------------+-------------+------------+---------------+
| Rafael Nadal    |       16496 |      18858 |      87.47500 |
| Jimmy Connors   |       13689 |      16652 |      82.20600 |
| John McEnroe    |        9855 |      12089 |      81.52000 |
| Guillermo Vilas |        7328 |       9845 |      74.43400 |
| Alex Molcan     |         452 |        623 |      72.55200 |
| Goran Ivanisevic |        6559 |       9401 |      69.76900 |
| Thomas Muster   |        4070 |       6117 |      66.53600 |
| Andres Gomez    |        3434 |       5192 |      66.14000 |
| Marcelo Rios    |        2712 |       4127 |      65.71400 |
| Henri Leconte   |        4706 |       7233 |      65.06300 |
+-----------------+-------------+------------+---------------+
10 rows in set (1 min 11.880 sec)
```

d) ***Top 5 Jogadores em Piso Duro (Hard Ground)***

```
mysql> SELECT
    ->     p.Name,
    ->     COUNT(*) AS WinningGames
    -> FROM
    ->     player p
    -> JOIN
    ->     games g ON p.Name = g.winner
    -> WHERE
    ->     g.Ground = 'Hard'
    -> GROUP BY
    ->     p.id_player, p.Name
    -> ORDER BY
    ->     WinningGames DESC
    -> LIMIT 5;
+----------------+--------------+
| Name           | WinningGames |
+----------------+--------------+
| Roger Federer  |          801 |
| Novak Djokovic |          658 |
| Ti Chen        |          609 |
| Andre Agassi   |          608 |
| Yen-Hsun Lu    |          562 |
+----------------+--------------+
5 rows in set (1.003 sec)
```